

上下文工程 (Context Engineering)

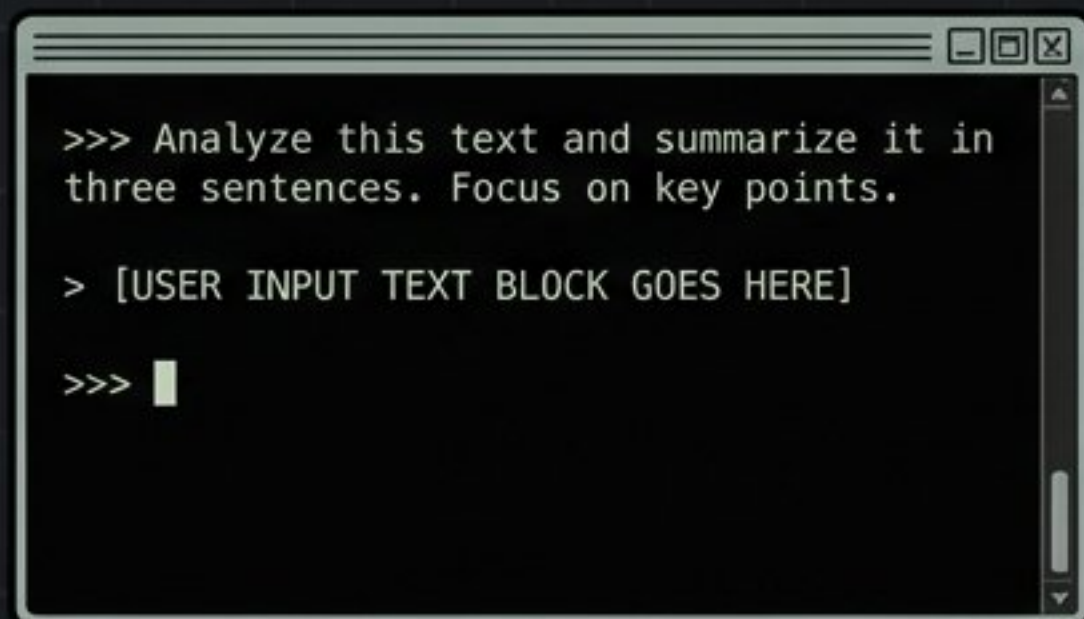
成本、速度与状态的终极战场 —— 从 Prompt 调优到管道工程的范式转移



“CONTEXT ENGINEERING IS THE NEW PROMPT ENGINEERING.”

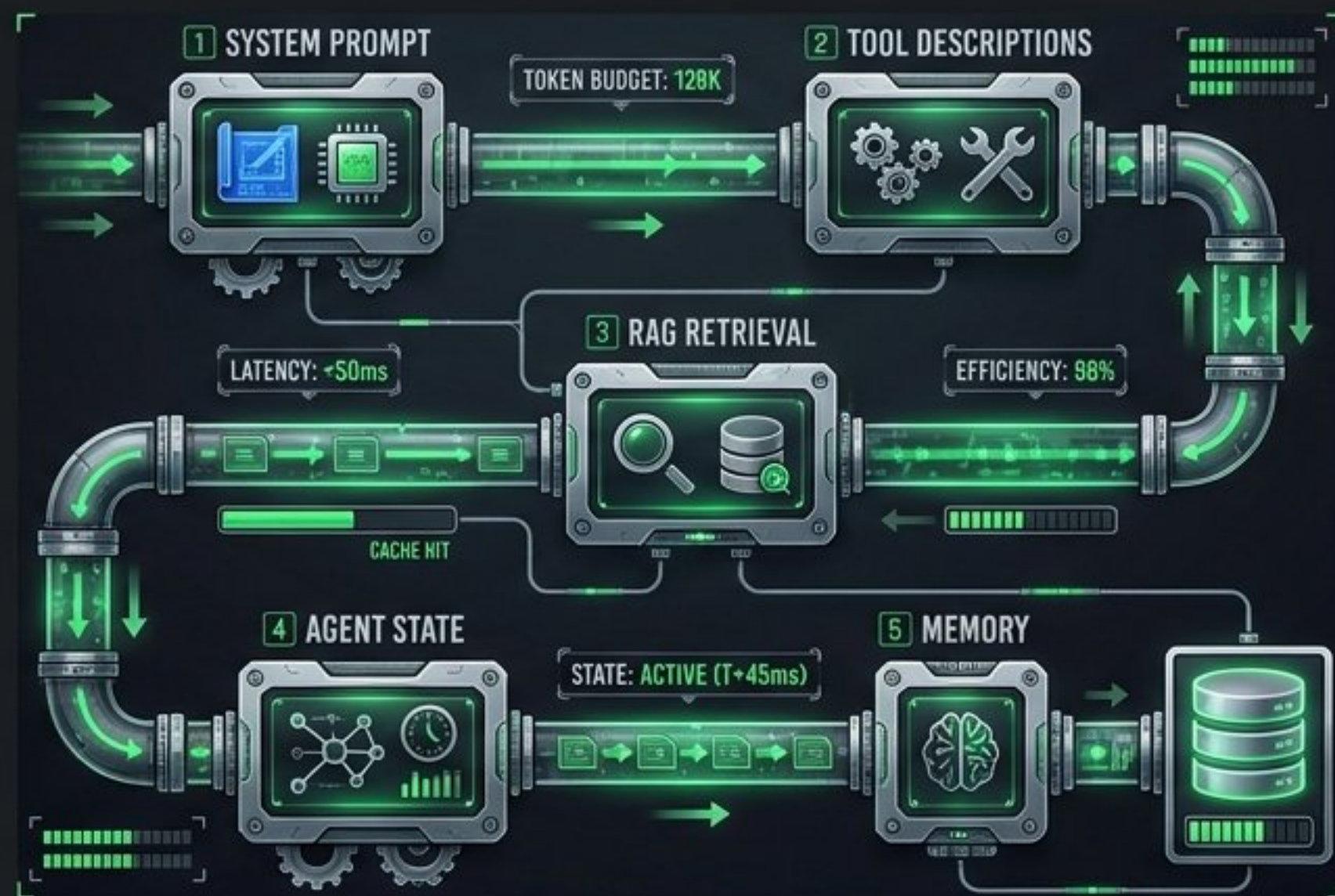
— Andrej Karpathy (2024)

2023（手工时代）



单纯的语言艺术

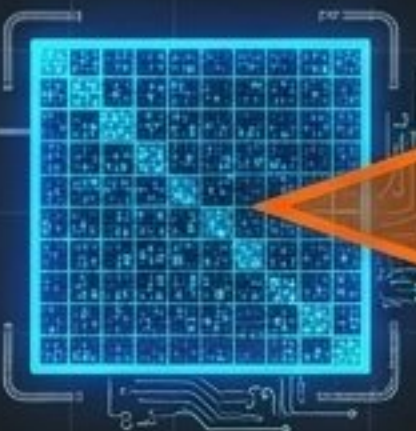
2026（工程时代）



系统级 Token 预算分配

揭开幻象：上下文不是文本框， 而是一笔随长度爆炸的物理账单

4K Context



$O(n^2)$

增加 32 倍上下文长度 = 增加 1024 倍计算量

GPT-4o 计费表：

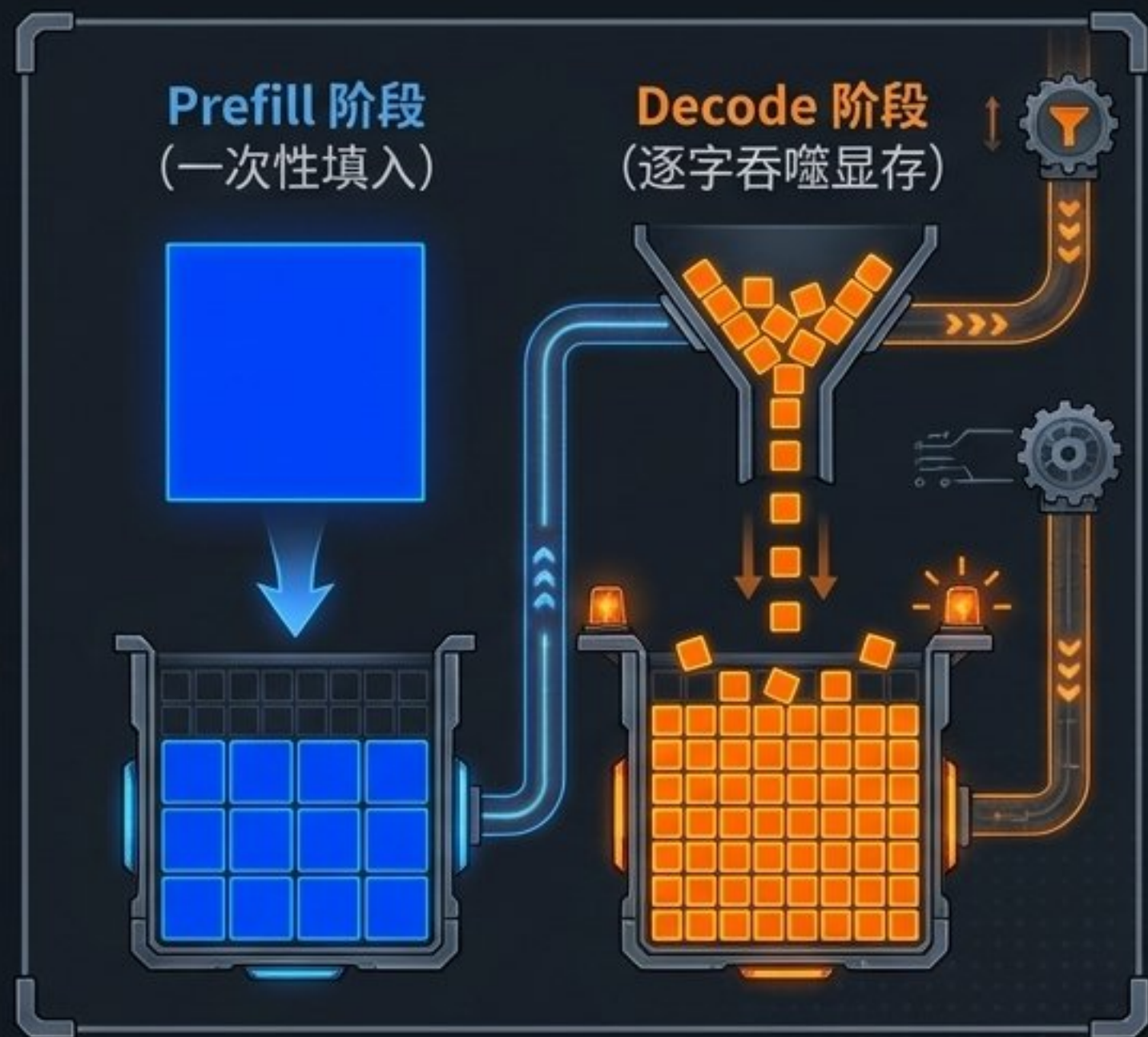
输入：\$2.50 / 1M Token

输出：\$10.00 / 1M Token

警示： 输出 token 更贵，因为每个输出都需要完整的前向传播计算。

128K Context

第一道物理墙：被严重低估的 KV Cache 内存黑洞



工程警示：单 GPU 处理满窗口的 128K 输入往往受限于 KV Cache 显存，而非模型本身。

破局策略一：Prompt Caching 斩断重复计算，成本直降 90%



10万次/天的业务账单对比

→ 无缓存 (No Cache): $21K \times \$3/M$

\$6,300/天

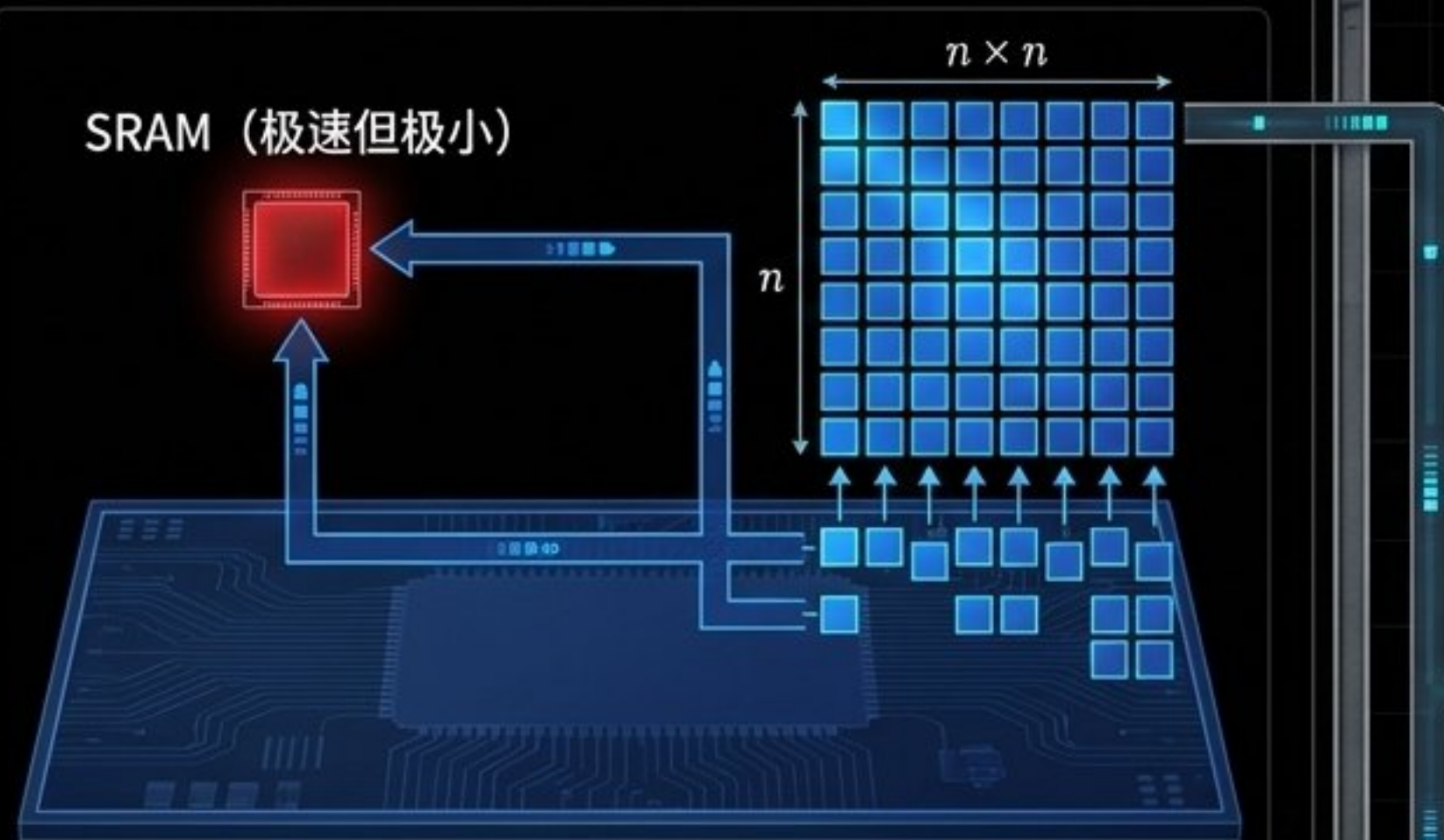
→ 有缓存 (Cached): $20K \times \$0.3/M + 1K \times \$3/M$

\$900/天

(节省 86%)

破局策略二：榨干底层硬件极限 (FlashAttention & PagedAttention)

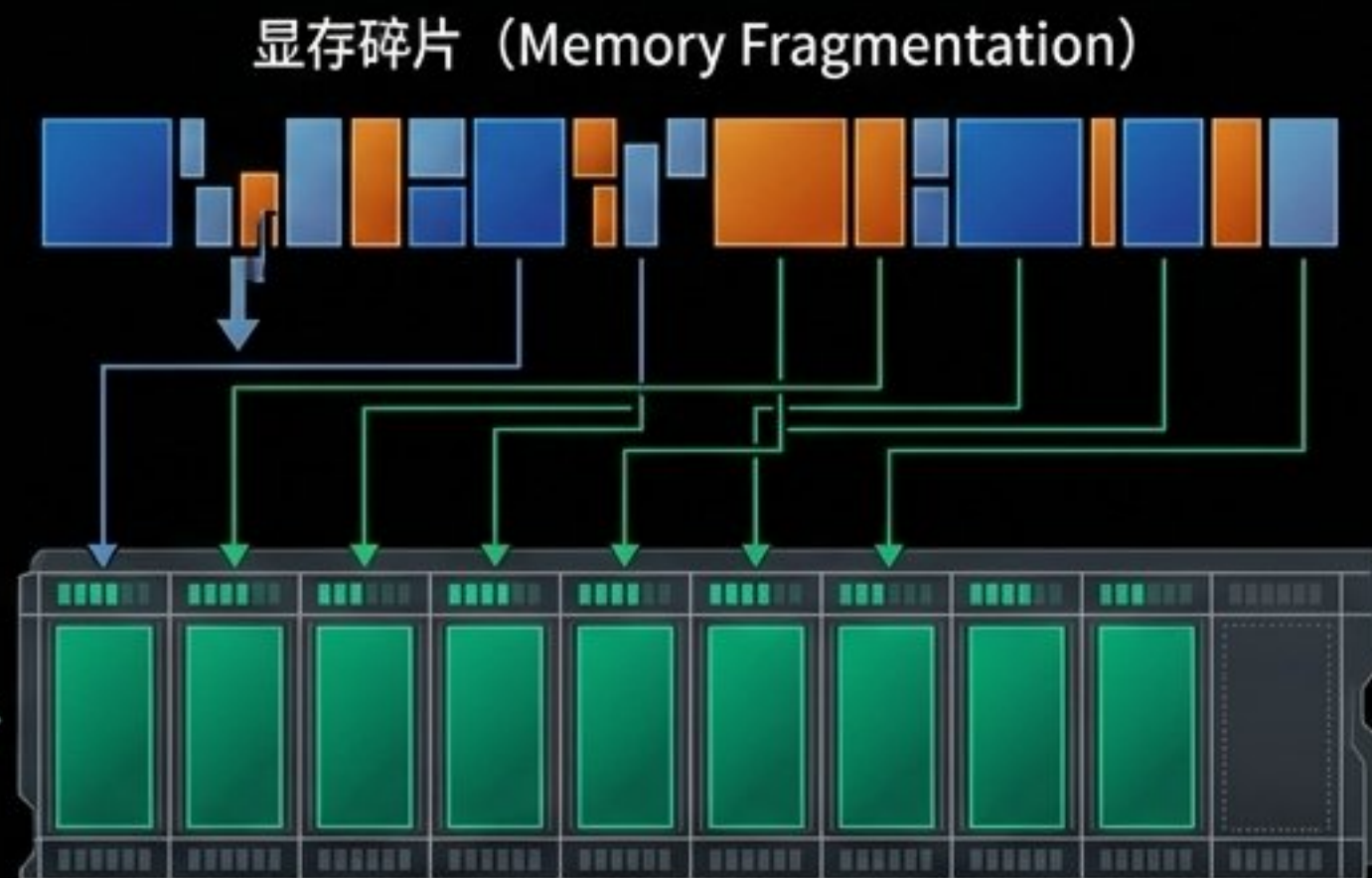
FlashAttention: IO 感知调度



HBM (巨大但较慢)

打破内存带宽瓶颈，分块直达片上缓存

PagedAttention: 内存分页机制



OS physical page table

消除显存碎片，吞吐量提升 2-4 倍

破局策略三：投机解码 (Speculative Decoding) 的双引擎加速



加速比 $\approx (1 + \text{接受率} \times k) / (1 + \text{小模型开销})$

数学保证：输出分布与纯大模型生成完全等价 (近乎免费的 2-3 倍提速)

现实的引力：“宣称窗口”不等于“有效窗口”

Llama 4 Scout

10M 标称 (Nominal)

~512K 有效 (Effective)

GPT-5 / Claude 4.5

1M 标称

~256K 有效

受限于训练数据分布，
切勿在生产环境逼近物理
极值。超出有效长度，
准确率呈断崖式下跌。

失忆地形图：模型为何在中间迷失（Lost in the Middle）



架构对决：长上下文（Long Context）vs. 检索增强生成（RAG）

长上下文 (< 100K)



整本生吞

	高推理成本
	高首字延迟 (TTFT)
	存在 Lost in the Middle 盲区

适合：强关联逻辑分析

楚河汉界：200K – 500K
决策边界

Cost

Latency

Scale

Accuracy

RAG (> 1M)



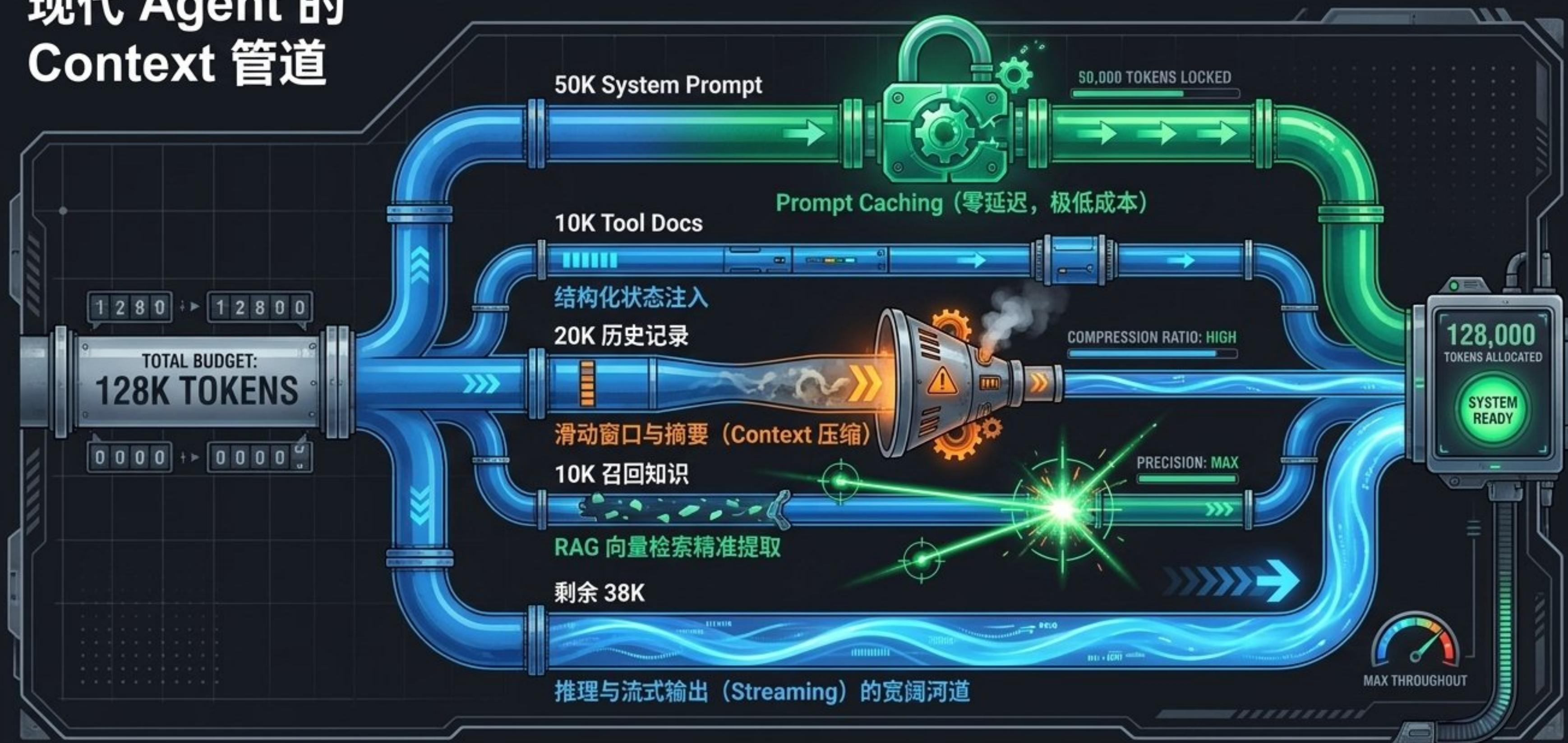
精准手术刀

	极速检索
	低成本、低延迟
	防干扰能力强

适合：海量数据的大海捞针

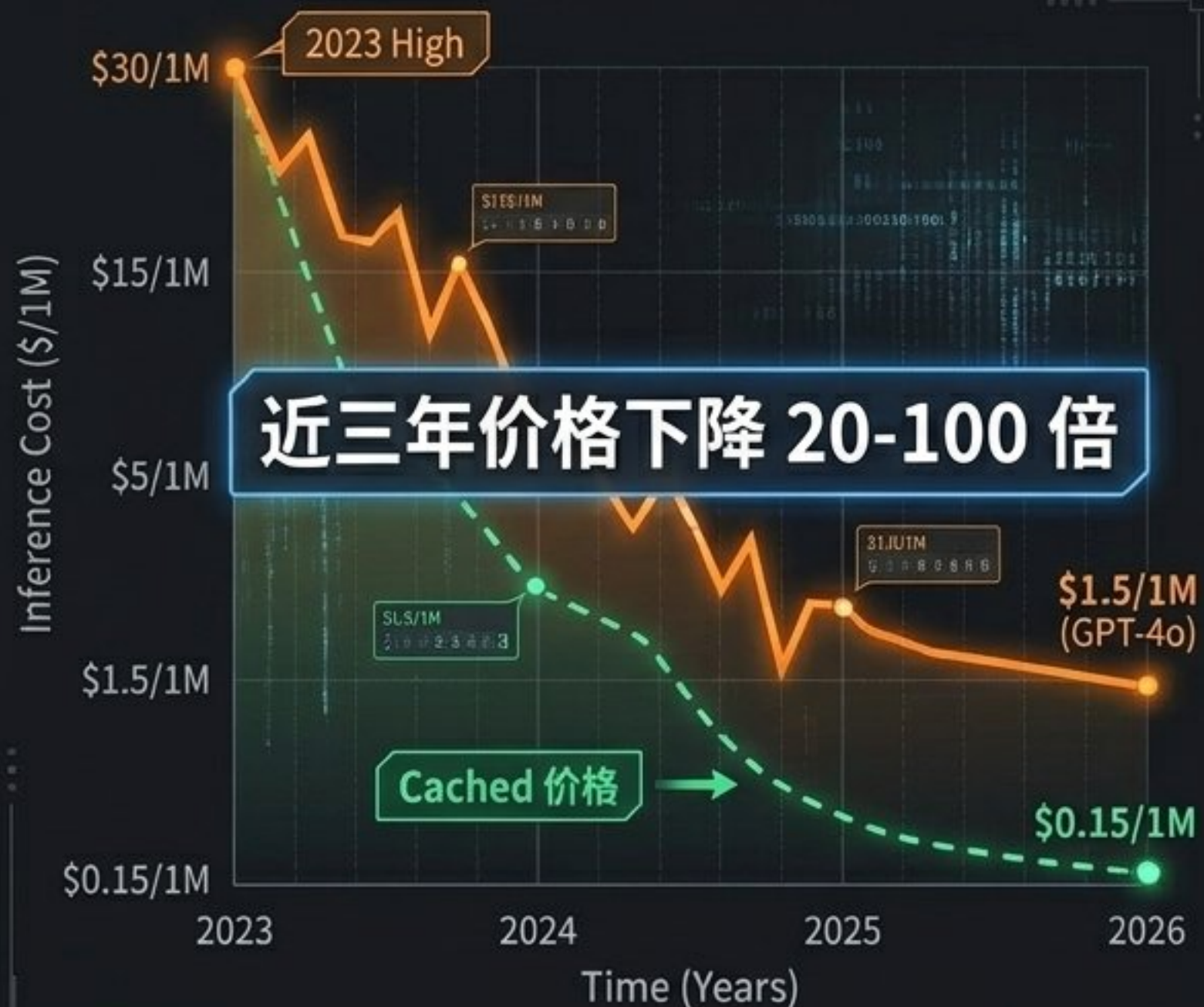
综合演练：设计一个现代 Agent 的 Context 管道

用显存换取时间，用前缀复用斩断二次方成本，用分级架构打破 IO 瓶颈。

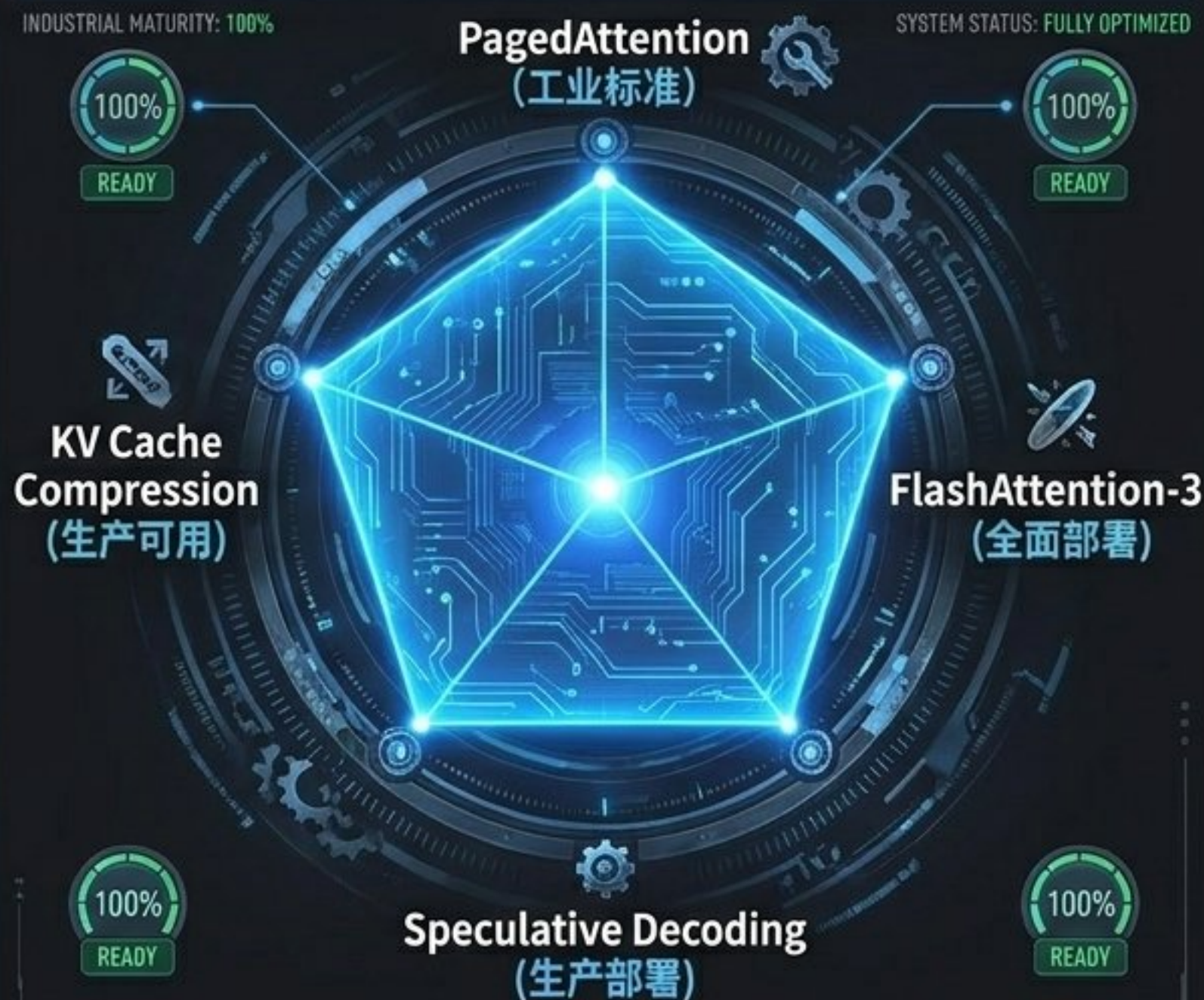


2026 技术快照：代际成本暴跌与基础设施标配化

代际成本暴跌 (Cost Crash)



基础设施标配化 (Infrastructure Standardization)



终局总结：质量、成本、速度的不可能三角 (The Trinity)

质量 (Quality)

QUALITY: OPTIMIZED

上下文压缩 (Summarization)

牺牲微小质量，换取成本与速度大幅下降。

KV Cache

牺牲空间成本，绝对保障速度。

Prompt
Caching

COST: MINIMIZED

成本
(Cost)

SPEED: MAXIMIZED

速度
(Speed)

保质量、降成本、提速度的完美一击

失败模式警告：认清物理边界（Reality Check）



缓存失效陷阱

频繁微调 System Prompt 导致 Cache 命中率归零，账单瞬间暴涨 10 倍。



盲信标称长度

在百万上下文中跑跨代码仓等多跳推理，导致模型逻辑链断裂崩溃。



全量硬塞（拒用 RAG）

将百万条业务数据直接塞入 Context 导致破产且命中率极差。大海捞针测试好看，不代表真实业务能用。

文档向量空间

第 23 讲：Embedding 与向量检索

—— AI 的记忆系统

无论窗口多大，也装不下 10 亿文档。如何在大海中精准捞针？